

Part B — Answer Outlines

A 6–8 point skeleton for every past or likely long question, with the diagram/table to draw. Marks shown are as printed in the paper. Use with the unit notes (§ numbers point to them).

How to use: write the numbered points as headings/first lines, expand each with 1–2 sentences, draw the named figure with labels. **IMPORTANT** = asked in a paper. **DRAW:** = diagram to include. **Marks guide:** 16 marks Part B question = typically 2 sub-parts; spend ~1.2 min per mark. Paper names: SEE Feb'26 (*SPA SEE 2022.pdf*), Model QP, CIE-1 Dec'24, CIE-2 Jan'25, CIE-1 Nov'25, CIE-2 Dec'25, CIE-3 Jan'26.

UNIT 1 — Q2 (compulsory)

1.1 Streaming pattern for collecting data, with example **IMPORTANT** SEE Feb'26 2a (8M) ·

CIE-1 Dec'24 3a (5M)

1. Define: stream pattern = collection tier **connects to a stream source and pulls data in**; one request → continuous responses (Unit 1 §4.5).
2. **DRAW:** Fig 2.9 bottom (stream source ⇌ service, "Response (continuous)") next to request/response optional.
3. Roles flipped: service becomes the client; vs request/response where an external client pushes.
4. Characteristics: asynchronous, multiple chunks, non-blocking.
5. Example: Meetup RSVP JSON stream over one long-lived HTTP connection / connect to Twitter for sentiment analysis.
6. "Ingest a stream and produce another stream": analysis output is a new stream.
7. Scaling: direct persistent connection leaves nodes idle ⇒ buffering layer, no business logic (Figs 2.11, 2.12).
8. When to use: live updates, large datasets in chunks, video streaming.

1.2 Publish/subscribe pattern with diagram **IMPORTANT** SEE Feb'26 2b (8M) · CIE-1 Dec'24 2a

(6M) · CIE-1 Nov'25 PartA

1. Definition: producers publish to a broker by topic; broker delivers to all subscribed consumers; asynchronous decoupling.
2. **DRAW:** Fig 2.7 (4 producers → Messages → Broker with Topic A/B/C and Subscriptions → Consumers).
3. 6 flow steps (publish, sort by topic, subscribe, broker routes, deliver to all subscribers, async).
4. Component table: publisher, subscriber, topic, broker, message, subscription.
5. Push vs pull delivery by product; producer need not subscribe.
6. Benefits: decoupling, scalability from 4 cars to all cars; broker can add persistence/reliability.
7. Example: Fig 2.8 traffic topic — cars publish "current traffic", a car subscribes.
8. Drawbacks: broker is critical; delivery guarantees depend on product.

1.3 Streaming data architecture for a scenario **IMPORTANT** Model QP 2a (10M) · CIE-1 Dec'24

1a (10M vehicle tracking) · CIE-1 Nov'25 1a (10M IoT)

1. State the blueprint (Fig 1.5) — **DRAW**: five tiers + long-term storage + clouds for clients, labelled with scenario devices.
2. Map each tier to the scenario (Unit 1 §8 table: collection, queue, analysis, in-memory, data access, storage).
3. Interaction pattern justification (stream / one-way / pub-sub; request/ack where a token is needed).
4. Scaling model: horizontal, stateless collection nodes behind a load balancer, buffering layer.
5. Security: authentication + authorization at each tier, TLS in transit (Fig 1.6).
6. Fault tolerance: logging (RBML/SBML/HML), replicated durable queue, checkpoints.
7. Example flow of one message end-to-end (sensor → alert).

1.4 Importance of scaling **IMPORTANT** Model QP 2b (6M) · CIE-1 Dec'24 A3

1. Define scaling; two types.
2. **DRAW/TABLE**: vertical vs horizontal (restaurant vs highway).
3. Vertical limit = machine capacity.
4. Horizontal: add nodes, elastic/auto-scaling, goal for every tier.
5. Stateless patterns scale easily (load balancer, Fig 2.10).
6. Stream pattern needs buffering layer (Figs 2.11–2.12).
7. Benefits: throughput, availability, cost.

1.5 Fault tolerance in the collection tier: checkpointing, RBML, SBML, HML

IMPORTANT CIE-1 Dec'24 3b (5M) · CIE-1 Nov'25 1a · Model QP 1.2

1. Why: nodes fail; client data not reproducible; goal = no loss, recover as if no crash.
2. Checkpointing: global snapshot; possible loss after last snapshot; hard across layers.
3. Logging idea: replayable messages ⇒ consistent state without a snapshot.
4. **DRAW**: Fig 2.15 + 2.16 (RBML happy path + recovery).
5. SBML: **DRAW** Fig 2.17 (log before send, ACK, delete).
6. Comparison table (what logged / protects / recovery / cost).
7. HML: single storage, asynchronous writes, 43% less overhead.
8. Jointly: restore checkpoint, replay log after it (CIE scheme).

1.6 Real-time classes; overlap with streaming systems **IMPORTANT** CIE-1 Nov'25 2a (5M) ·

CIE-1 Dec'24 A1 · Model QP 1.1

1. Table 1.1 (hard/soft/near: examples, latency, tolerance).
2. Overlap: all need timely processing; soft/near blurry.
3. Hard: strict, failure = disaster.
4. Soft: deadlines preferred (video streaming); near: minimal lag (stock trading).
5. Streaming data system: non-hard real-time computation + clients consume when needed (Fig 1.4).
6. Conclusion: streaming relaxes strict deadlines (see trap box).

1.7 Compare interaction patterns **IMPORTANT** CIE-1 Nov'25 2b (5M)

1. List five patterns.
2. Request/response + half/full-async.
3. Request/acknowledge + token example.
4. Pub/sub + topic example.
5. One-way (lossy, telemetry).
6. Stream (continuous pull).
7. **TABLE:** response, sync/async, use, reliability (Unit 1 §4.6).

1.8 Sender-based vs receiver-based logging **IMPORTANT** CIE-1 Dec'24 3b (5M)

1. Definitions.
2. **DRAW:** Fig 2.14 with "we don't want to lose this" on both sides.
3. RBML: raw input, before processing, protects the node.
4. SBML: processed output, before sending, protects against next tier failure; needs ACK/delete.
5. Recovery steps for each.
6. Cost/overhead; HML compromise.

UNIT 2 — Q3 / Q4

2.1 Need for message queuing tier with example **IMPORTANT** Model QP 3a (10M) · CIE-2

Dec'25 1a (6M) · CIE-1 Dec'24 2b (4M)

1. Architecture without the queue (Fig 3.2) — tiers tightly coupled.
2. Decoupling of tiers/components.
3. Buffering surges / back-pressure (**DRAW** Fig 3.6 A→B→C).
4. Offline/slow consumers; durable storage.
5. Delivery guarantees, ordering, flow control.
6. Fault tolerance (data centre link cut).
7. Example: San Diego collection / Amsterdam analysis; or traffic app.

2.2 Producer, broker, consumer with a queuing layer **IMPORTANT** SEE Feb'26 4b (6M) · CIE-

2 Dec'25 2a (10M)

1. **DRAW:** Fig 3.5 (blueprint with producer → broker(queue) → consumer).
2. Producer sends message to broker.
3. Broker places it in a queue (abstraction over multiple queues).
4. Consumer reads from the broker.
5. Isolation: producer faster than consumer, durable messaging, offline consumers.
6. Delivery semantics + acknowledgements.
7. (10M) failure points of Fig 3.12 + security of Fig 3.14.

2.3 Build a message-queueing model for the traffic-replay product **IMPORTANT**

Model QP 3b (6M) · CIE-1 Dec'24 5b (4M logistics tracking)

1. Problem: transient queue discards consumed messages (Fig 3.10) — no history.

2. Choose a **durable** message queue (disk/JDBC/file).
3. **DRAW:** Fig 3.11 — real-time consumer (zero delay) + offline consumer $\Rightarrow$ historical data store.
4. Retain messages by city/day (topic or key); replay by re-reading.
5. Sizing/retention for week/month replay.
6. Security and replication of durable store.

2.4 Message delivery semantics for that scenario **IMPORTANT** Model QP 4b (6M) · SEE Feb'26

3a(iv) · CIE-1 Dec'24 A5

1. Define three semantics.
2. Pick exactly-once (history must be complete and not double counted) or at-least-once + dedup.
3. **DRAW:** Fig 3.13 — producer (no blind retry) $\rightarrow$ broker $\rightarrow$ consumer + metadata in persistent storage.
4. Technique 1: verify with broker instead of retrying.
5. Technique 2: store last message ID/offset.
6. Bonuses: auditing, duplicate detection.
7. Trade-off table (Table 3.1: complexity vs guarantee).

2.5 Seven points of failure in stream processing, with example **IMPORTANT** Model QP

4a (10M) · CIE-2 Jan'25 3a (10M) · SEE Feb'26 8b (10M) · CIE-1 Dec'24 4a · CIE-1 Nov'25 3a

1. **DRAW:** Fig 4.15 numbered 1–7.
2. Points 1–2: incoming stream and its network — handle gracefully.
3. Point 3: stream processor — manager restarts/moves it.
4. Points 4–5: output connection and destination — control back-pressure.
5. Points 6–7: streaming manager (headless) and application driver.
6. Consolidate: data loss + loss of resource management.
7. Techniques: replication + coordination; state-machine ($k+1$) vs rollback recovery.
8. Real example: fraud detection / ad billing (exactly-once needed). For "communication failures": checkpoint + log replay, idempotent jobs, acknowledgements. For "broker crash": replication/ISR, ack after write, consumers resume from committed offset, leader election.

2.6 Fault-tolerance challenges in the queuing tier: broker crash, inter-broker network, drive failure **IMPORTANT** SEE Feb'26 3b (8M)

1. **DRAW:** Fig 3.16 (3 brokers with storage).
2. Broker crash: in-memory messages at risk $\Rightarrow$ producer waits for disk-write ack.
3. Replicate to > 1 broker; hold little data in memory (performance trade).
4. Network disrupted: replicated data safe; broker rejoins and re-syncs; mid-send message risk.
5. Questions: new replica? delay before "down"? never restored?
6. Drive failure: replicas of lost data? un-flushed data lost? recover broker?
7. Impact on reliability: loss / duplicates $\Rightarrow$ choose delivery semantics.

2.7 Scenario: Coke vending machines **IMPORTANT** SEE Feb'26 3a (8M)

1. Impact of interruption: lost revenue/offers, wrong stock recommendations.
2. Days of data tolerable: depends on inventory handling/locale; small.
3. Historical data: yes, for reporting and inventory trends.
4. Delivery semantics: at-least-once (justify: duplicate offer harmless, missed offer costs sale; at-most-once arguable).
5. Architecture: machines → collection (one-way/pub-sub) → durable queue → analysis (geo + stock) → push/tweet.
6. Draw small blueprint.

2.8 Apache Samza architecture for a professional networking platform

IMPORTANT SEE Feb'26 4a (10M) · CIE-2 Jan'25 A1 (Samza diagram) · CIE-3 Jan'26 A1

1. Samza = stage-wise framework on YARN + Kafka.
2. **DRAW:** Unit 2 §7.2 figure + Fig 4.10 (Yarn client → resource manager → task runners ↔ Kafka).
3. Kafka input topics partitioned (events/sec, scalability).
4. YARN allocates containers; one task per partition.
5. Local state + Kafka changelog (state consistency, low latency).
6. Output topic to personalisation service (job chaining).
7. Failure: YARN restart + changelog replay; Kafka replication.
8. Semantics: at-least-once + idempotent updates.

2.9 Distributed stream-processing architecture + key components **IMPORTANT** CIE-3

Jan'26 1a (10M: need 2 + diagram 2 + explanation 6) · Model QP 1.4

1. Need: single machine cannot handle volume/velocity (1 TB/hour turbine).
2. **DRAW:** Fig 4.4.
3. Application driver.
4. Streaming manager.
5. Stream processors.
6. Data sources / output store.
7. Map to Spark/Storm/Flink/Samza (one line each + Fig 4.7–4.10).
8. Example: turbine monitoring (Fig 4.5).

2.10 Stream-processing frameworks — explain any two with diagram **IMPORTANT**

CIE-3 Jan'26 4 (10M) · CIE-2 Jan'25 2a · SEE Feb'26 1.4

1. List: Spark Streaming, Storm, Flink, Samza.
2. Pick two (suggest Storm + Flink, or Spark + Storm).
3. **DRAW:** Fig 4.8 / Fig 4.9 (or 4.7).
4. Components and roles (Nimbus/supervisors, JobManager/TaskManagers).
5. Processing model (tuple-at-a-time vs stream-first dataflow; micro-batch).
6. Unit of work: topology vs job.
7. Windowing/delivery support (Tables 5.2–5.3).

8. When to use.

2.11 DBMS vs streaming systems (query management) **IMPORTANT** CIE-2 Jan'25 1a (6M) ·

CIE-3 Jan'26 A2

1. **DRAW:** Fig 4.3 (query to data vs data through query).
2. One-time vs continuous query.
3. Pull vs push of results.
4. Changing data during downtime.
5. Query state after crash.
6. Active/passive inversion; Table 4.1.

2.12 State management; state-machine vs rollback recovery **IMPORTANT** CIE-3 Jan'26

2a (4M), 3b (4M)

1. State needed for aggregations/joins/late events.
2. Continuum (Fig 4.13) in-memory → replicated persistent store.
3. Ad impression/click join example (Fig 4.14).
4. State-machine: replicate job, same ordered input, $k+1$ resources, quick failover.
5. Rollback: checkpoint + log replay, lower overhead, slower recovery.
6. Table with use cases (intrusion detection vs moderate latency).

UNIT 3 — Q5 / Q6

3.1 Window type for treadmill energy/temperature **IMPORTANT** SEE Feb'26 5a (10M)

1. Requirements: independent, non-overlapping, resets each minute.
2. **Temporal tumbling window, 1 minute** (trigger = eviction = 1 min).
3. **DRAW:** timeline with windows [0–1), [1–2)... drained at each trigger (Unit 3 §2.5 figure).
4. Metric 1: sum power consumption per window, global.
5. Keep full messages so future split by gym location is possible.
6. Metric 2: filter temperature > threshold; alert; window resets.
7. Why not sliding (overlap double-counts); stream vs event time.
8. Framework: Flink built-in (Table 5.3).

3.2 Fitbit dashboard — sliding window **IMPORTANT** Model QP 5a (10M) · CIE-2 Jan'25 4a (10M)

1. Need: average speed per age group, refreshed every 5 s over last 30 min.
2. **Sliding window: length 30 min (eviction), interval 5 s (trigger)**.
3. **DRAW:** Fig 5.3 relabelled (30 min / 5 s) with overlapping windows.
4. Group by age band (12–17, 18–24, 25–34...).
5. Stream time vs event time + time skew.
6. Framework support (Table 5.2): Flink both times + custom policies; Spark stream time only; Storm/Samza need work.
7. Justify vs tumbling (would not overlap/refresh every 5 s).

3.3 Sliding vs tumbling **IMPORTANT** SEE Feb'26 6a (6M)

1. Definitions with trigger/eviction.
2. **DRAW:** Fig 5.3 and Fig 5.5 side by side.
3. Overlap vs non-overlap.
4. Policies (time-based vs count/time).
5. Types.
6. Use cases; framework support. Use the comparison table (Unit 3 §2.4).

3.4 Summarization techniques **IMPORTANT** SEE Feb'26 5b (6M), 6b (10M) · CIE-3 Jan'26 Q5 (10M) ·

CIE-2 Jan'25 1b

1. Why: unbounded, no memory, approximate acceptable.
2. Classify: sampling, distinct count, frequency, membership.
3. Reservoir sampling — k/n replacement (Fig 5.6).
4. HyperLogLog — hash, register index, leading zeros, harmonic mean (Fig 5.7).
5. Count-Min Sketch — hash per row, increment, min (Figs 5.8–5.9).
6. Bloom filter — set bits, AND to test, false positives only (Fig 5.10).
7. Example for each (ad network).
8. For "any two": spend ~5 marks each with diagram + steps + example + error type.

3.5 Reservoir sampling for 10 M ad views per minute **IMPORTANT** Model QP 5b (6M)

1. Problem: cannot hold in memory/disk; need random sample.
2. Fixed reservoir of k items filled with first k .
3. n -th item enters with probability k/n .
4. If chosen replaces a random reservoir element.
5. Equal probability for all items.
6. **DRAW:** Fig 5.6 with reservoir 15 and 16th item (15/16).
7. Implementable in any framework; example ad analysis.

3.6 Caching strategies **IMPORTANT** Model QP 6a (10M) · CIE-2 Jan'25 5a (10M, any three)

1. Caching system = in-memory key-value store (DRAM).
2. Read-through (Fig 6.4).
3. Refresh-ahead (Fig 6.5).
4. Write-through (Fig 6.6).
5. Write-around (Fig 6.7).
6. Write-back (Fig 6.8).
7. **TABLE** with pros/cons; products (Memcached write-around, EHCache, Hazelcast, Redis).
8. Risk: dual writes, node loss $\Rightarrow$ IMDB alternative.

3.7 Three ways to write data to long-term storage **IMPORTANT** CIE-3 Jan'26 2b (6M), 3a (6M

video analytics)

1. **DRAW:** Fig 6.2 (A, B, C).
2. A: message-by-message at stream speed.

3. B: batch in analysis tier.
4. A/B risk: store cannot keep up.
5. C: queue + batch loader (decoupling).
6. Batch loader features/tools (Gobblin, Secor).
7. Choose C for CCTV analytics (compliance, model training).

3.8 Bloom filter for membership queries **IMPORTANT** CIE-2 Jan'25 1b (4M)

1. Membership query = "has X been seen before?"
2. Bit array + hash functions.
3. Insert sets bits; **DRAW** Fig 5.10.
4. Query ANDs bits; any 0 $\Rightarrow$ never seen.
5. False positives, no false negatives.
6. Real use: blocklist check before expensive lookup.

UNIT 4 — Q7 / Q8 (full notes in the existing Unit 4 PDF)

Question	Where asked	Skeleton pointer
Commits and offsets; two commit strategies	SEE Feb'26 7a (10M)	Unit 4 notes — C6
Three ways of sending messages	SEE Feb'26 7b (6M)	Unit 4 notes — B3
Range assignor 6 partitions	SEE Feb'26 8a (6M) · CIE-1 Nov'25 5a	Unit 4 notes — C5; see trap box in Quick Revision §4
E-commerce fraud: failures, communication failures, broker crash	SEE Feb'26 8b (10M)	Use Unit 2 §2.5 + §2.6 (seven points, broker crash) and Unit 1 §1.5 (checkpoint/logging) — this "Unit 4" question is really Unit 2 content
Kafka ecosystem diagram; why Kafka; producer program; JSON consumer program	CIE-1 Nov'25, Model QP	Unit 4 notes — A6, A5, B9, C9

UNIT 5 — Q9 / Q10

5.1 Why Kafka Connect for MySQL + PostgreSQL + warehouse; features; limitations **IMPORTANT** SEE Feb'26 9a (10M)

1. Systems are existing datastores you cannot modify $\Rightarrow$ Connect with ready connectors, configuration only.
2. **DRAW:** Connect architecture (Unit 5 §3): source $\rightarrow$ workers/tasks/converters $\rightarrow$ Kafka $\rightarrow$ sink.
3. Built-in: config management, offset storage, parallelism, error handling, data types, REST API.
4. Scalability: tasks.max, distributed workers.
5. Fault tolerance: worker failure $\Rightarrow$ reassignment; offsets/configs in Kafka topics; dead-letter queue.

6. CDC (Debezium) for accurate change capture.
7. Limitation 1: complex business logic / joins (SMTs stateless) $\Rightarrow$ custom consumer or Kafka Streams.
8. Limitation 2: no connector / need fine control or app-embedded events $\Rightarrow$ custom producer/consumer.

5.2 Criteria for choosing Connect vs custom producers/consumers **IMPORTANT** [SEE](#)

Feb'26 10b (8M) · CIE-1 Dec'24 5a

1. **TABLE:** can you modify the source code? $\Rightarrow$ clients; datastore you can't change $\Rightarrow$ Connect.
2. Effort: configuration vs coding.
3. Built-in features: offsets, parallelism, error handling, REST.
4. Data formats via converters.
5. Transformation needs: SMT vs Streams.
6. Operational overhead of a Connect cluster.
7. Latency/control needs.
8. Example: MySQL $\rightarrow$ Kafka via Connect; order events from your own app via producer.

5.3 Kafka pipeline for logs from microservices **IMPORTANT** [SEE Feb'26 9b \(6M\)](#)

1. **DRAW:** Unit 5 §3.2 diagram.
2. Services $\rightarrow$ source connector/shipper $\rightarrow$ app-logs topic.
3. Kafka as buffer, retention, replication.
4. Kafka Streams filter/enrich.
5. Sink connectors $\rightarrow$ Elasticsearch, S3.
6. SMT (TimestampRouter, MaskField); dead-letter queue.
7. Security and monitoring.

5.4 Considerations for a Kafka-based fraud-detection pipeline **IMPORTANT** [CIE-2](#)

Dec'25 Q3 (10M) · Model QP 9a

1. Timeliness (ms decisions).
2. Reliability / delivery guarantees.
3. Throughput and scalability.
4. Data formats and schema evolution.
5. ETL vs ELT.
6. Security / access control.
7. Failure handling (dead letters, replay).
8. Coupling and agility. **TABLE** per consideration with fraud example.

5.5 Table–table and streaming joins, with example **IMPORTANT** [CIE-2 Dec'25 Q4 \(10M\) ·](#)

Model QP 10b

1. Stream–table: enrichment from local table copy via CDC (Fig 14-7).
2. Table–table: non-windowed, current state of both, equi-join (same key and partitioning) or foreign-key join.
3. Streaming (windowed) join: match same key within time window (Fig 14-8).

4. **DRAW:** Fig 14-7 and Fig 14-8.
5. Examples: click enriched with profile; search ↔ click within 1 s.
6. Requirements: same partition count/key; same task; RocksDB window state.
7. Join types: inner/left/outer where applicable.

5.6 Processing with local state **IMPORTANT** SEE Feb'26 10a (8M) · Model QP 1.10

1. Definition (given statement).
2. **DRAW:** Fig 14-4 (trades topic partitions → processors with local state → aggregated topic).
3. Example: min and average stock price per day (store min, sum, count).
4. Partition by key so each instance holds its subset.
5. Issues: memory, persistence, rebalancing.
6. Kafka Streams: RocksDB + changelog topic (log compaction).
7. Local vs external state comparison.

5.7 Single-event processing pattern — order-confirmation e-mail **IMPORTANT** Model

QP 10a

1. Pattern: process each event in isolation (map/filter).
2. **DRAW:** Fig 14-3 adapted: orders topic → processor → confirmation-email action.
3. Order event (order ID, customer ID, products, total) consumed.
4. Validate/enrich/map to e-mail message (stateless).
5. Send e-mail; optionally produce to "confirmations" topic.
6. No state ⇒ easy recovery/scaling; another instance can take over.
7. Implementable with a plain consumer or Kafka Streams.

5.8 Kafka Connect — connectors, types, SMTs **IMPORTANT** Model QP 9b · CIE-2 Dec'25 A4, A5

1. Connector = plug-in deciding number of tasks and splitting work.
2. Two types: source / sink.
3. Tasks do the data movement; workers run connectors/tasks.
4. Workers handle REST config, offset commits, failure reassignment.
5. Converters (JSON/Avro/Protobuf).
6. SMT list; RegexRouter and TimestampRouter definitions.
7. **DRAW:** Connect architecture figure.

5.9 Kafka Streams architecture / word count / stock stats / click-stream

IMPORTANT syllabus · SEE Feb'26 1.10

1. Library, no external cluster; topology = DAG of source/processor/sink nodes.
2. **DRAW:** Fig 14-10 (word count topology).
3. Tasks = partitions (Figs 14-11/14-12); repartition topic (Fig 14-13).
4. Word count: flatMapValues → map → filter → groupByKey → count.
5. Stock stats: 5 s window advancing 1 s, aggregate min/count/avg.
6. Click stream: stream-table leftJoin + stream-stream windowed join.
7. Failures: changelog rebuild, standby replicas, offsets.